Each time through the `while` loop, we will start with turning on the old pin, delay, then turn on the new pin, delay, and finally turn off the old pin. This is what will allow us to not only make the primary colors, but also the intermediate colors, as well. The one color that will not work this way is white. To get around that we use the following block of code:

```
if (newPin == 0) {
  for (int i=0; i<3; i++) digitalWrite(rgb[i], HIGH);
  delay(time);
  for (int i=0; i<3; i++) digitalWrite(rgb[i], LOW);
}
```

This `if` statement will turn on all of the colors of the LED to make white, delay, and then turn them all off, using that neat little trick from earlier to step through each pin in a `for` loop. This happens so fast that it simply appears that each color turns on or off at the exact same time.

The last thing to do in our `while` loop is to check if the state of our switch has changed at all by running our debounce code again, and if the state has changed, the `while` loop will exit. Finally, the last thing in our sketch is a line to make sure that all of the colors are off if the switch has not been triggered, and then we add a short delay just to make sure our readings stay in line.

Now, much of that may or may not have made a whole lot of sense. Don't worry too much about that for now because we will spend the rest of the chapter trying to explain how these control structures work in much greater depth. You might want to come back to this and see if it makes more sense after reading through the rest of this chapter.

Comparative and Logical Operators

Conditional statements like the `for` loop and `if` statement discussed in this chapter use comparison and logical operators to test for conditions. **Comparative operators** are used to compare the left side of an expression against the right side to produce a value of true or false. With this piece of information, the Arduino can use it to make decisions depending on the type of control structure being used. So if you wanted to test whether or not a variable held a particular value, you might use an expression like the following:

```
myValue == 5
```

Here we use the `==` comparative operator to test if the variable or value on the left of the operator is equal to the variable or value on the right. In this case, the expression would return as true if `myValue` contained the value 5, and false if it did not. You should note that we use the phrase "is equal to" instead of "equals". This helps us to differentiate the comparison operator (`==`) from the assignment operator (`=`), with the former referred to as "is equal to" and the later "equals".

Table 4-1 lists other comparative operators that we will discuss in greater depth and provide examples for in this chapter and beyond.